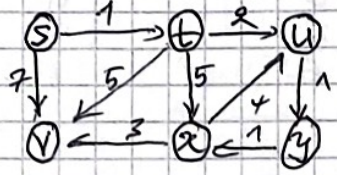
76250043

DUSHIME MUDAHERA RICHARD

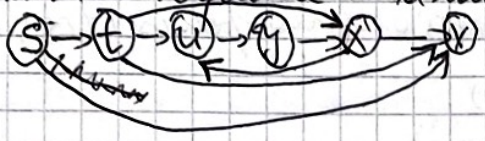
GRAPHS ALGORITHM : HOMEWORK 1.

1) $D = (V, E)$.



a) Is (s, t, u, y, x, v) a topological sort of D?

⇒ No, this is not a topological sort because for (s, t, u, y, x, v) to be a topological sort every directed edge must go from an earlier vertex (v) in the sequence to a later vertex



the Vertex x → u violates the topological ordering

b) Does the Digraph D admit a topological sort?

⇒ No, D = (V, E) does not admit the topological sort because a directed graph admits a topological sort if it is a DAG (Directed Acyclic graph) means no cycle

→ the following edges (u → x → y → u) forms a cycle

c) Find a tree of shortest paths from vertex s to all other vertices in D. using Dijkstra algorithm.

Notation : d(v) : shortest distance estimate from s to v.
Π(v) : predecessor of v in the shortest path Tree
S : set of vertices.
Q : priority Queue.

* Initialisation

$S = \emptyset$, $Q = \{s(0), t(\infty), u(\infty), v(\infty), y(\infty), x(\infty)\}$.

Iteration 1:
Edges s → t (weight 1)
New distance d(s) + w(s,t) = 0 + 1 = 1
Edges s → v (weight 7)
New d = d(s) + w(s,v) = 0 + 7 = 7

* Iteration 2 : Extract t.
min from Q : t : d(t) = 1
- processing all outgoing edges from t.
- Edge t → u (w = 2)
New d : d(t) + w(t → u) = 1 + 2 = 3.
current d(u) = ∞
check : 3 < ∞ ? yes update d(u) = 3

* Edge t → x (w = 5)
New d d(t) + w(t, x) = 1 + 5 = 6.
current d(x) = ∞
check 6 < ∞ ? yes
update d(x) = 6.

* Edge t → v (w = 5)
New d = d(t) + w(t, v) = 5 + 1 = 6.
current d(v) = 7
check 7 < 6 ? No update.
d(v) = 6.

Q : {u(3), v(6), x(6), y(∞)}

Iteration 3 : Extract Vertex u.
min from Q : u : d(u) = 3
Add S : S = {s, t, u}.
- processing all outgoing edges from u
* Edge u → y w = 1
New d d(u) + w(u, y) = 3 + 1 = 4
current d(y) = ∞
check 4 < ∞ ? yes. update d(y) = 4

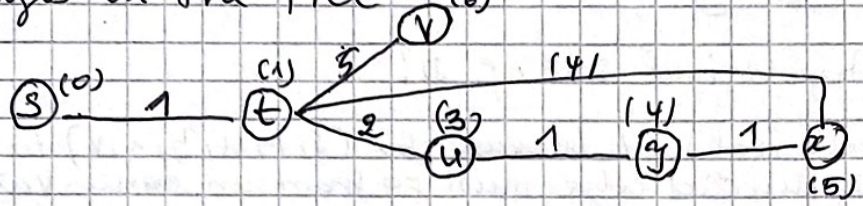
Iteration 4, Extract vertex y.
min Q : y : d(y) = 4
Add to S : S = {s, t, u, y}
outgoing edges from y.
* Edge y → x w = 1.
current y, d(y) = 4
New d : d(y) + w(y → x) = 4 + 1 = 5.
current d(x) = 6.
check 5 < 6 ? yes update d(x) = 5.

* Iteration 5 : Extract x
current d(x) = 5.
- edges from x
(x, v) : d(v) = min (6, 5 + 3) = 6 (no change)

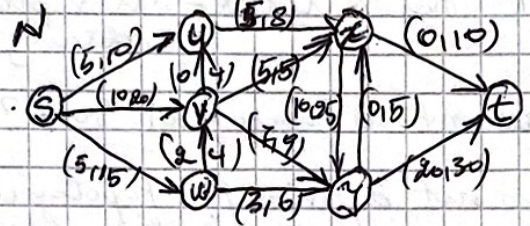
Iteration 6:
Current x (dx) = 6.
No outgoing edges from x.
$\Delta = \phi$.
Termination.

v	Distance from s	Predecessor	Shortest path
s	0	—	s
t	1	s	$s \to t$
u	3	t	$s \to t \to u$
y	4	u	$s \to t \to u \to y$
x	5	y	$s \to t \to u \to y \to x$
v	6	t	$s \to t \to v$.

Edges in the Tree:



2/ Network $N = (D = (V, E), c, s, t)$ with ordered pairs $(f(e), c(e))$ on the edges:



a)/ 4 s,t-cuts in the network.
→ For each s,t-cut $C = \delta^+(X)$. (Specify the cut edges) the capacity $cap(C)$ and net outflow $f(\delta^+(X)) - f(\delta^-(X))$.

Cut 1: $X_1 = \{s, u\}$
- cut edges: $(s,v), (s,w), (u,x), (v,u)$
  $cap(C_1) = 20 + 15 + 8 + 4 = 47$
  outflow $f(s,v) + f(s,w) + f(u,x) + f(v,u) = 10 + 5 + 5 + 0 = 20$
  inflow $f(w,v) = 2$.
  Net outflow $20 - 2 = \underline{18}$

Cut 2: $X_2 = \{s, v\}$
Cut edges: $(s,u), (s,w), (v,x), (v,y)$
$cap(C_2) = 10 + 15 + 5 + 9 = 39$
outflow: $f(s,u) + f(s,w) + f(v,x) + f(v,y) = 5 + 5 + 5 + 7 = 22$.
Inflow: $f(w,v) = 2$.
Netflow: $22 - 2 = \underline{20}$.

Cut 3: $X_3 = \{s, w\}$
Cut edges: $(s,u), (s,v), (w,x), (w,y)$
$cap(C_3) = 10 + 20 + 4 + 6 = 40$.
outflow: $f(s,u) + f(s,v) + f(w,x) + f(w,y) = 5 + 10 + 2 + 3 = 20$
Inflow: none (edges from $(v,w), (y,w)$ are not present.
Net outflow: $20 - 0 = 20$.

Cut 4: $X_4 = \{s, u, v, w\}$
cut edges: $(u,x), (v,x), (v,y), (w,y)$
~~outflow~~ $cap(C_4) = 8 + 5 + 9 + 6 = \underline{28}$
outflow $= 5 + 5 + 7 + 3 = 20$.
Inflow: none
Net outflow $= \underline{20}$

4) Residual Network and 4 Augmenting path:

* For each edge (a,b):
  Forward edge: if $f(a,b) < c(a,b)$, add (a,b) with capacity $c(a,b) - f(a,b)$
  Backward edge: if $f(a,b) > 0$ add (b,a) with capacity $f(a,b)$

Table of Residual capacities:

Edge	Forward	Backward
(S,U)	5	5
(S,V)	10	10
(S,W)	10	5
(V,U)	4	0
(U,X)	3	4
(V,X)	0	3
(V,Y)	2	7
(W,V)	2	0
(W,Y)	3	3
(X,T)	10	0
(Y,T)	10	20
(X,Y)	15	10
(Y,X)	5	0

Augmenting paths & Max flow increase:

Path 1: S → U → X → T
Residuals S→U: 5, U→X: 3, X→T: 10
max flow increase min(5,3,10) = 3

Path 2: S → V → Y → T
Residuals: S→V: 10, V→Y: 2, Y→T: 10
max flow increase min(10,2,10) = 2

Path 3: S → W → Y → T
Residuals: S→W: 10, W→Y: 3, Y→T: 10
Max flow increase: min(10,3,10) = 3

Path 4: S → V → U → X → T
Residuals: S→V: 10, V→U: 4, U→X: 3, X→T: 10
Max flow increase: min(10,4,3,10) = 3

3) DIGRAPH CONNECTIVITY
Input: A digraph $D=(V,E)$ and a positive integer K.
Question is D K-connected?

a) A digraph $D=(V,E)$ is K-connected if: 1. $|V| > K$.
2. for every 2 distinct vertices $u,v \in V$ such that $(u,v) \notin E$ there exist K internally vertex-disjoint directed u,v-paths.

* To prove that the problem is NP we check the certificate
- For instance (D,K) where $D=(V,E)$ is K-connected, the certificate is:
  Certificate = $\{P_{u,v} \mid u,v \in V, u \neq v, (u,v) \notin E\}$
  Where each
  $P_{u,v}$ is a collection of K directed paths from u,v to x:
  $P_{u,v} = \{P_1^{u,v}, P_2^{u,v}, \ldots, P_K^{u,v}\}$

Each path $P_i^{u,v}$ is represented as a sequence of vertices:
$P_i^{u,v} = (v_0 = u, v_1, v_2 \ldots v_\ell = v)$
Where $(v_j, v_{j+1}) \in E$ for all $j = 0,1,\ldots \ell-1$.

* Verification Algorithm
- Input: Digraph $D=(V,E)$, integer K and certificate as described above
1) For each pair (u,v) with $u \neq v$ and $(u,v) \notin E$:
  Step 1.1: check exactly K paths are provided in $P_{u,v}$.
  " 2: For each path $P_i^{u,v} = (v_0, v_1 \ldots v_\ell)$:
    Verify $v_0 = u$ and $v_\ell = v$
    Verify each edge $(v_j, v_{j+1}) \in E$ for $j = 0 \ldots \ell-1$
    Verify the path is simple (no repeated vertices)
  Step 1.3: For each pair of distinct paths $P_i^{u,v}$ and $P_j^{u,v}$ where $i \neq j$:
    Let internal($P_i^{u,v}$) be the set of vertices in $P_i^{u,v}$ excluding u, v
    Verify internal($P_i^{u,v}$) $\cap$ internal($P_j^{u,v}$) $= \emptyset$
If all checks pass accept otherwise reject.

Time Complexity
- Number of pairs: At most $\binom{|V|}{2} = O(|V|^2)$

For each pair:
- K paths to check
- Each path has length at most $|V|$
- checking one path: $O(|V|)$
- checking internal disjointness for $\binom{K}{2}$ pairs of path $O(K^2 \cdot |V|)$

Total time $O(|V|^2 \cdot K^2 \cdot |V|) = O(K^2 \cdot |V|^3)$

Since K is given as part of the input, this is polynomial time in the input size

$\Rightarrow$ Digraph connectivity $\in NP$.

b/ Polynomial-time algorithm using Menger's theorem and maximum flow to solve this problem

- Mengers theorem (vertex version for Dygraphs)
For distinct vertices u, v in a Digraph D, the maximum number of internally vertex-disjoint u, v-paths equals the minimum size of a vertex cut separating u from v
- This can be computed using max-flow by converting the vertex-disjoint paths problem into an edges-disjoint paths problem via vertex splitting

— Algorithm —
- Input: Digraph $D = (V, E)$ and positive integer K.
- output: YES if D is K-connected: NO otherwise

* Steps:

1: check if $|V| > K$, if not, return NO

2, For each pair of distinct vertices $u, v \in V$ such that $(u, v) \notin E$:

2:1 construct auxiliary network $N_{u,v}$.
- Start with Digraph D
- Vertex splitting: For each vertex $w \in V \setminus \{u, v\}$:
  - Replace w with 2 vertices: w^{in} and w^{out}
  - Add directed edge (w^{in}, w^{out}) with capacity 1
  - For each edge $(x, w) \in E$: redirect it to (x, w^{in})
  - For each edge $(w, y) \in E$: redirect it to (w^{out}, y)
- Keep vertices u and v unsplit
- All edges except the splitting edges (w^{in}, w^{out}) have capacity ∞

Step 2:2: Compute the Maximum flow
- Use the edmonds-karp algorithm (or any max-flow algorithm) to compute the maximum flow from u to v in $N_{u,v}$

let $f_{max}(u, v)$ denote this max flow value

Step 2:3 check flow value
if $f_{max}(u, v) < K$, return No
(there are fewer than K internally vertex disjoint paths from u-v

Step 3: if all pairs pass the test return Yes

* Correctness: By menger's theorem and the vertex splitting construction:
- the max flow value $f_{max}(u, v)$ in the auxiliary Network $N_{u,v}$
= the max number of internally vertex disjoint u, v-paths in D
- D is K-connected if and only $f_{max}(u, v) \geq K$ for all relevant pairs (u, v)

* Time complexity: $O(|V|^2) \times O(|V| \cdot |E|^2) = O(|V|^3 \cdot |E|^2)$
this is polynomial time in size $(|V|, |E|, K)$.

$\Rightarrow$ Digraph connectivity can be solved in polynomial time with complexity $O(|V|^3 \cdot |E|^2)$.

DUSHIME MUDAHERA RICHARD 76250043.

3/ Time Complexity Analysis

Step 1: $O(1)$
Step 2: For each pair (u, v):
Number of pairs: At most $|V|^2$
Step 2.1 (Vertex splitting):
- creating auxiliary network: $O(|V|+|E|)$
- Auxiliary network has:
  - $O(|V|)$ vertices (each original vertex split into 2, + u and v)
  - $O(|V|+|E|)$ edges

Step 2.2 (max-flow computation using Edmonds-Karp):
- Time complexity: $O(|V'| \cdot |E'|^2)$ where $|V'| = O(|V|)$ and $|E'| = O(|V|+|E|)$
- Since $|E| \geq |V| - 1$ for connected graphs $O(|V| \cdot |E|^2)$

Total time complexity $O(|V|^2) \times O(|V| \cdot |E|^2) = O(|V|^3 \cdot |E|^2)$
this is polynomial in the input size $(|V|, |E|, K)$.

Conclusion: Dygraph connectivity can be solved in polynomial time with complexity $O(|V|^3 \cdot |E|^2)$.

4/ Graph G with matching M^* and vertex cover C^* such that $|C^*| = |M^*|$

a) is M^* maximum and C^* minimum?
= No, because the condition $|C^*| = |M^*|$ doesnot guarantee optimality by König-Egervary theorem this equality holds for maximum matching and minimum vertex cover only in bipartite graphs.
eg: Triangle K_3 has maximum matching of size 1 but minimum vertex cover of size 2. so $|M_{max}| \neq |C_{min}|$

b) Does C^* contain exactly one endpoint of each edge in M^*?
⇒ No, because A vertex cover must have atleast one endpoint from each edge, but it can have both endpoints, the property of exactly one endpoint per matching edge holds only in bipartite graphs when M^* and C^* are both optimal

5/ a) Equivalence with isolated vertices

Given: $(G = (V, E), K)$ with $K \leq |V| / 3p = |V| - 2K, G' = (V', E')$ where $V' = V \cup V_0$ with $|V_0| = P$ isolated vertices.
To prove G has independent set of size $\geq K$ if G' has independent set of size $\geq |V'|/2$.
Note: $|V'| = |V| + P = |V| + (|V| - 2K) = 2|V| - 2K$, so $|V'|/2 = |V| - K$.

Proof: ⇒ Suppose G has independent set S with $|S| \geq K$.
- in G', S is still independent (no new edges added b/n orginal V
- Add all P isolated vertices $S' = S \cup V_0$
- S' is independent in G' (isolated vertices has no edges)
- $|S'| = |S| + P \geq K + (|V| - 2K) = |V| - K = |V'|/2$

⇐ Suppose G' has independent set S' with $|S'| \geq |V'|/2 = |V| - K$
- Let $S'_{orig} = S' \cap V$ (Vertices from orginal graph)
- Let $S'_{iso} = S' \cap V_0$ (isolated vertices)
- Since $|V_0| = P = |V| - 2K$ and $|S'| \geq |V| - K$:
  $|S'_{orig}| = |S'| - |S'_{iso}| \geq (|V| - K) - (|V| - 2K) = K$.

S'_{orig} is independent in G' and thus in G. Therefore G has independent set of size $\geq$

b/ Half-independent set is NP-complete

Problem: Input: Graph $G = (V, E)$
Does G have independent set S with $|S| \geq |V|/2$?

* Proof of NP-completeness:
- Step 1: Half-independent set $\in$ NP
Certificate: An independent set with $|S| \geq |V|/2$.
Verificate: check S is independent (no edges within S) and $|S| \geq |V|/2$ (this is polynom)

- Step 2: Reduction from one-third independent set:
- Given instance $I = (G = (V, E), K)$ of one-third-independent set with $K \leq |V|/3$:
1/ compute $P = |V| - 2K$
2/ Create $G' = (V', E)$ where $V' = V \cup V_0$ with $|V_0| = P$ isolated
3/ Return $f(I) = G'$ as instance of Half-independent set

* Correctness by part (a):
G has independent set $\geq K \Leftrightarrow G'$ has independent set $\geq |V'|/2$.

Polynomial time: Adding $P \leq |V|$ isolated vertices takes $O(|V|)$ time

$\Rightarrow$ Half-independent set is NP-complete.

c/ Let $G = (V, E)$ have perfect matching M and independent set S
claim: $|S| \geq |V|/2$ if and only if S contains exactly one endpoint from each edge in M.

Proof:
- Each edge must contribute one vertex to S or else $|S| < |V|/2$
- More than one vertex per matching edge is not independent, since edges connect those endpoints

d/ polynomial-time Algorithm via 2-SAT

For graphs with a perfect matching, Half-independent set can be solved in polynomial time:
-1/ Find perfect matching M (using standard polynomial matching)
-2/ For each edge $e = (u_e, v_e)$ in M, create a boolean variable x_e: x_e = True if $u_e \in S$, false if $v_e \in S$.

3/ For every edge (a, b) not in M:
- For matching edges containing a and b add a 2-CNF clause to prevent both a and b in S (using literals for their positions as endpoints)

4/ the constructed problem is a 2-SAT instance (polynomial-time solvable)

5/ If 2-SAT is satisfiable such independent set exists